Scaling Techniques

How the numeric fields were scaled, with code (Jira T7-32)

Project: Customer Churn Analysis, Project Team 7
Prepared by: Harjot Kaur, Data Engineer
Date: 11 November 2026
Notebook: `Data_Preparation/preprocessed_data/T7-31_to_T7-33_Clean_Encode_Scale_Split_Handover.ipynb`

---

1 Purpose

This document explains which scaling technique was used to prepare the churn data, why it was chosen over the alternatives, how data leakage was prevented, and how the saved scaling values can be reused. Every code snippet below is taken from the preparation notebook, so the document and the files in the repository match.

2 Why the Data Needs Scaling

After encoding, ten of the twelve features are 0/1 values, but `tenure` and `MonthlyCharges` are measured on much larger and different scales (Table 1). Both methods in this project are sensitive to this. K-Means groups customers by Euclidean distance, so an unscaled field with a wide range would dominate the clusters (Han et al., 2012). An artificial neural network (ANN) trains by gradient descent, which converges faster and more reliably when inputs are on similar scales (Géron, 2022).

Table 1
Numeric Fields Before Scaling (All 7,043 Customers)

Field	Unit	Min	Max	Mean	SD	Median	Skewness	IQR outliers
tenure	months	0	72	32.37	24.56	29	0.24	0
MonthlyCharges	dollars	18	119	64.76	30.09	70	-0.22	0
Binary / one-hot (10)	0 or 1	0	1					

3 Scaling Techniques Considered

Table 2
Comparison of Scaling Techniques

Technique	Formula	Strengths	Weaknesses	Fit for this data
Standardisation (z-score), `StandardScaler`	z = (x - mean) / SD	Centres at 0 with SD 1; suits distance-based and gradient-based methods; easy to reverse	Sensitive to extreme outliers	**Chosen.** No outliers and low skew
Min-max normalisation,	x' = (x - min) / (max - min)	Bounded 0 to 1; matches 0/1 features	One extreme value squeezes the rest;	Possible, but less stable for new data

Technique	Formula	Strengths	Weaknesses	Fit for this data
`MinMaxScaler`			new values can fall outside 0 to 1	
Robust scaling, `RobustScaler`	x' = (x - median) / IQR	Not affected by outliers	Less common in ANN practice; no benefit without outliers	Not needed: 0 outliers
Log transform	x' = log(x + 1)	Reduces strong right skew	Changes the shape of the data; harder to explain	Not needed: skew is low

4 Technique Chosen and Why

Standardisation with scikit-learn's `StandardScaler` was chosen (Pedregosa et al., 2011). Both numeric fields have low skewness (0.24 and -0.22) and no outliers under the 1.5 x IQR rule, so the main weakness of standardisation does not apply and robust scaling or a log transform would add nothing. Standardisation gives both fields a mean of 0 and a standard deviation of 1, which puts them on a comparable footing with each other for K-Means and gives the ANN well-centred inputs (Géron, 2022; James et al., 2021). The saved mean and standard deviation also make the scaling easy to reverse when results are explained to the business.

Only `tenure` and `MonthlyCharges` were scaled. The ten binary and one-hot columns were left as 0/1, because they are already on a small, meaningful scale and scaling them would make their values harder to read (Kuhn & Johnson, 2013). `Churn` is the target and is never scaled.

5 Preventing Data Leakage

If the scaler were fitted on all 7,043 rows before splitting, the mean and standard deviation would include the validation and test customers, and the model would indirectly see information it is later tested on. This is data leakage, and it makes results look better than they really are (Kuhn & Johnson, 2013). To prevent it, the data was split first, the scaler was fitted on the training set only, and the same fitted values were then applied to the validation and test sets.

6 Code Snippets

6.1 Setup

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
SPLIT_DIR = '../train_test_sets'    # Data_Preparation/train_test_sets
PREP_DIR = '.'                      # Data_Preparation/preprocessed_data
```

6.2 Split first (stratified 70/15/15)

```
X = enc.drop(columns='Churn'); y = enc['Churn']
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=RANDOM_STATE)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=RANDOM_STATE)
```

6.3 Fit on training data only, then apply to all three sets

```
num_cols = ['tenure', 'MonthlyCharges']
scaler = StandardScaler().fit(X_train[num_cols])   # learns training mean and SD only

def scale(frame):
    out = frame.copy()
    out[num_cols] = scaler.transform(frame[num_cols])
    return out

train = scale(X_train).assign(Churn=y_train)
val   = scale(X_val).assign(Churn=y_val)
test  = scale(X_test).assign(Churn=y_test)
```

6.4 Save the scaled sets and the scaler values

```
params = pd.DataFrame({'mean': scaler.mean_.round(4),
                       'std': np.sqrt(scaler.var_).round(4)}, index=num_cols)
for name, part in [('train', train), ('val', val), ('test', test)]:
    part.to_csv(f'{SPLIT_DIR}/ats_{name}.csv', index=False)
params.to_csv(f'{SPLIT_DIR}/scaler_params.csv')
```

6.5 Clustering file (scaled on the full dataset)

Clustering is unsupervised and has no test set, so for Dilina's K-Means work the scaler is fitted on all 7,043 customers. `Churn` stays in the file for profiling the clusters afterwards and must not be used as a clustering feature.

```
clus = enc.copy()
clus[num_cols] = StandardScaler().fit_transform(enc[num_cols])
clus.to_csv(f'{PREP_DIR}/ats_clustering_ready.csv', index=False)
```

6.6 Check the result

```
for name in ['train', 'val', 'test']:
    part = pd.read_csv(f'../train_test_sets/ats_{name}.csv')
    print(name, part[['tenure', 'MonthlyCharges']].agg(['mean', 'std']).round(3))
```

6.7 Reuse or reverse the scaling with the saved values

These functions let any team member scale new customer records in exactly the same way, or turn scaled values back into months and dollars when explaining results.

```
params = pd.read_csv('../train_test_sets/scaler_params.csv', index_col=0)

def apply_scaling(frame):
    out = frame.copy()
    for col in params.index:
        out[col] = (out[col] - params.loc[col, 'mean']) / params.loc[col, 'std']
    return out

def undo_scaling(frame):
    out = frame.copy()
    for col in params.index:
        out[col] = out[col] * params.loc[col, 'std'] + params.loc[col, 'mean']
    return out
```

7 Results

Table 3
Scaler Values Learned From the Training Set (`scaler_params.csv`)

Field	Mean	Standard deviation
tenure	32.3105	24.5833
MonthlyCharges	64.5901	30.1506

Table 4
Mean and Standard Deviation After Scaling

File	tenure mean	tenure SD	MonthlyCharges mean	MonthlyCharges SD
`ats_train.csv`	0.000	1.000	0.000	1.000
`ats_val.csv`	0.010	0.996	0.039	1.006
`ats_test.csv`	0.006	0.998	-0.002	0.980
`ats_clustering_ready.csv`	0.000	1.000	0.000	1.000

The training set has a mean of exactly 0 and a standard deviation of 1, as expected. The validation and test sets are very close but not exact, because they were scaled with the training values rather than their own. This small difference is the correct result and confirms that no leakage occurred. Reversing the scaling with the saved values returns the original whole months and dollars (maximum difference below 0.0001).

8 Points to Remember

- Do not scale `ats_train.csv`, `ats_val.csv` or `ats_test.csv` again; they are already scaled.
- Any new data must be scaled with `scaler_params.csv`, not with a newly fitted scaler.
- The clustering file uses full-data scaling and is for K-Means only, not for the ANN.

- If the cleaning decisions change (for example, removing the 302 duplicates), re-run the notebook so the scaler values and all files are rebuilt together.

References

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer.

Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.